CIVICCONNECT PROJECT: MILESTONE 2 v2

Prepared by Ayanda Mabena, Nompilo Mbense and Refilwe Segele

Contents

Contents 1
Document control 2
Requirements, Traceability, and Scope Lead 3
Problem and Business Need 4
Stakeholder Analysis 5
Scope Baseline 6
In-scope 6
Out-of-scope 6
Deferred scope 7
Functional Requirements 7
Non-functional Requirements 9
Requirements Traceability Matrix 9
CONSTRAINTS AND TRADE-OFFs ANALYSIS 11
Scope Constraints: 12
Schedule & Resource Constraints: 12
Quality & Security Constraints: 12
Trade-Off / Ripple Effect: 12
Initial Risk Register 12
Risk Item 1 (High Priority): 12
Risk Item 2: 12
Forward Engineering Considerations Register 13
Engineering Decision Log 13
Decision 1 (Made): 13
Decision 2 (Deliberately Deferred): 13
Repository and Team Governance 14
Github governance 15
Team Gorvenance 15

AI Compliance and controls 16
Milestone 2: Architecture, Technology & Initial Design Baseline 17
Architecturally Significant Requirements (ASR) 18
Logical architecture and physical deployment diagram 18
Database Schema Specification Tables 23
Design Pattern Architecture Decisions 23
Updated Risk Register 24
Forward Engineering Considerations 25
Architecture, Technology & Initial Design Baseline 26
Sign-off 27
Milestone 1 feedback and evidence 27
CONCLUSION 28
References 29

Document control

Version	Date	Authors	Changes	Approved by
1.0	8 September 2026	Ayanda Mabena, Nompilo Mbense, Refilwe Segele	Initial engineering foundation: requirements baseline, stakeholder analysis, scope, RTM, initial risk register, GitHub governance	Team
2.0	25 September 2026	Ayanda Mabena, Nompilo Mbense, Refilwe Segele	Added ASRs, architecture diagrams and ADR 01 (architecture pattern), ADR 02 (technology stack), data/persistence baseline (ADR 03), design pattern decisions (ADRs), initial backend implementation	Team

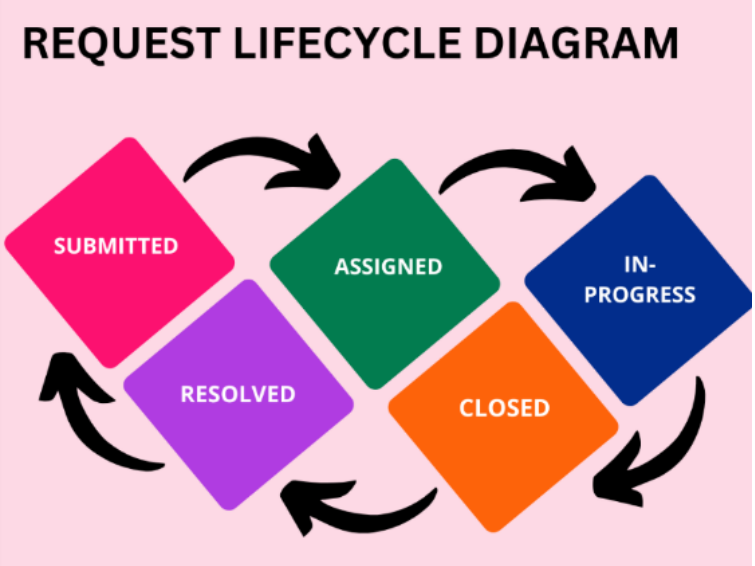
Requirements, Traceability, and Scope Lead

Problem and Business Need

The organisation manages service requests through a mix of communication methods, including phone calls, emails, WhatsApp messages, spreadsheets, and paper records. Because there is no single controlled record of a request's lifecycle, it is difficult to know where a request stands, who is handling it, what has been done, and whether it has been resolved. Complete tracking throughout a request's lifecycle is recognised as a core benefit of structured service request management, ensuring all parties stay informed of progress (ServiceNow, 2026) .

This fragmentation creates real frustration on both sides. Requesters often end up sending duplicate requests or following up across multiple channels just to get an answer, wasting their time. Staff face a similar problem: the same issue may surface across several platforms, forcing them to investigate or resolve it more than once. The result is a slower, more time-consuming process with delayed resolutions all round.

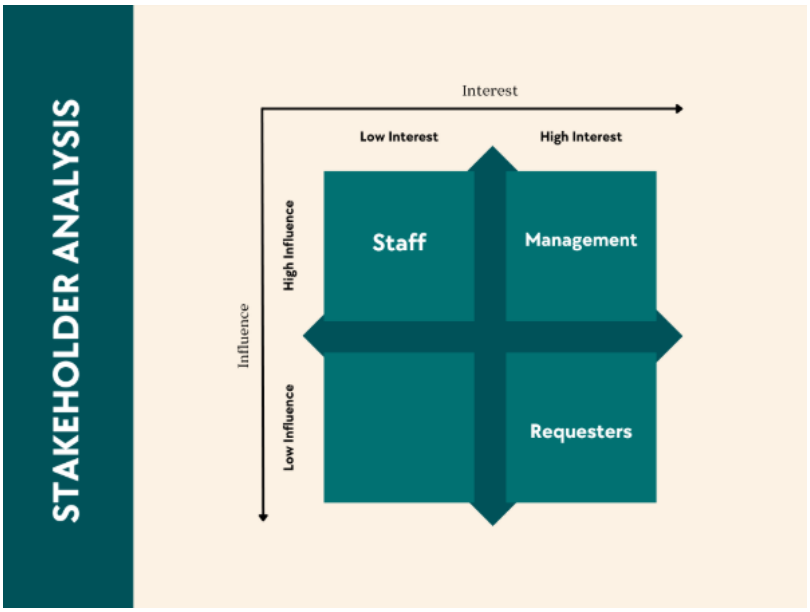
What the organisation needs is a system that allows people to submit and monitor requests, directly supporting the core business need to submit, manage, and monitor service requests in one place (ServiceNow, 2026) . Beyond that, the organisation needs clear ownership of every request. This includes who is responsible for it, what priority it carries, and what actions have been taken. Right now, that need isn't being met because accountability is weak for changes to request status and the actions taken. Solving this problem creates value across the organisation. Requesters gain visibility and clarity on the status of their requests. Staff save time and can delegate tasks more effectively and manage their workload with greater control. Management gains reliable, auditable data that supports accountability and service-performance reporting.



Stakeholder Analysis

Requesters need to be able to submit a new service request, categorise it, and view the status of their submitted requests. Staff need to be able to view requests and search, filter, or sort them, as well as assign or accept responsibility for a request. Management needs to be able to identify open, overdue, resolved, and closed requests.

One conflict identified is between Requesters and Staff. Requesters want the system to be fast and straightforward, allowing them to submit a request quickly and see results soon after. This may clash with Staff's needs, since resolving a request properly takes time, requires the Requester to provide sufficient detail upfront, and depends on staff having the capacity to investigate and act on it. A system optimised purely for speed on the Requester's side risks producing incomplete requests that are harder and slower for Staff to resolve. One functional requirement derived from this is that the system must allow a Requester to view, update, or delete their submitted request where appropriate.



Scope Baseline

In-scope

- Requesters can submit a new service request with relevant details.
- Requesters can categorise their request using a controlled category list, for instance, assign it to maintenance or IT support.
- Requesters can view the status of their submitted request.
- Requesters can view a list of their previously submitted requests.
- Requesters receive feedback when their request is accepted or completed.
- Staff can view requests relevant to them and search/filter/sort by useful criteria.
- Staff can view full request details.
- Staff can assign or accept responsibility for a request.
- Staff can update a request status from open to in progress and so forth.
- Staff can record actions, comments, or resolution notes on a request.
- Management can view service activity, including open, overdue, resolved, and closed requests.
- Management can view requests by category for basic reporting and accountability purposes.
- Role-based access so Requesters, Staff, and Management see only what's relevant to their role.

Out-of-scope

- Native mobile app — web-based only.

- Integration with external channels (auto-importing requests from WhatsApp, phone calls, or email into the system).
- Multi-language support.
- Advanced analytics or AI-driven prioritisation/auto-assignment of requests.
- Public API for third-party integration.
- Payment processing or billing functionality.

Deferred scope

- Automated notifications (SMS/emails) on status change.
- Advanced management dashboards/reporting beyond basic status counts,
- Public API for future integrations.

Automated notifications have been deferred, as this milestone prioritizes establishing the core, controlled lifecycle of a request. This includes submission, assignment, status tracking, and resolution. The system's primary aim is to be reliable and traceable, and this deferment protects that focus. Introducing a third-party notification service such as SMS or email would bring external dependencies, added cost, and integration risk that isn't justified until the core platform is stable. In line with the constraints, this decision protects the project's schedule and cost boundaries and reduces the risk of an unfamiliar third-party integration destabilising features more directly tied to the stated business need.

Functional Requirements

ID	Requirement	Source	Priority	Acceptance Criteria
FR-001	Requester can submit a new service request with relevant details.	Requester in scope	Must	Request will be saved using the ID and status “Submitted” once mandatory

				fields are completed.
FR-002	Requester can categorise their request.	Requester in-scope	Must	Submission will be blocked unless a category is selected.
FR-003	Requester can view the status of their submitted request	Requester in-scope	Must	Status updates visible to Requester in real time as Staff changes it.
FR-004	Requester can view a list of their previous requests	Requester in-scope	Should	“My Requests” list all past submissions with status and date
FR-005	Requester can view, update or delete their submitted requests	Stakeholder Analysis-Staff/Requester conflict	Should	Editing allowed only while status is “Submitted”, locked once staff accepts it.
FR-006	Staff can view requests relevant to them and search/filter/sort	Staff in scope	Must	Filters return only matching requests.
FR-007	Staff can view full request details	Staff in scope	Must	All submitted fields visible on one screen
FR-008	Staff can assign or accept responsibility for a request	Staff in scope	Must	Accepting sets status to “In progress” and records the staff member as the owner
Fr-009	Staff can update request status through	Staff in scope	Must	Submitted-> In progress-

	controlled transitions			>Resolved->closed
FR-010	Staff can record actions, comments or resolution notes on a request	Staff in scope	Should	Notes are timestamped
FR-011	Management can view service activity	Management in scope	Must	Counts of open or overdue data
FR-012	Management can view requests by category for basic reporting	Management in scope	Should	Category filter returns only matching requests
FR-013	System enforces role-based access so each role sees only what's relevant to them	Role-based access	Must	Requesters see only their own requests; staff cannot access Management reports

Non-functional Requirements

ID	Requirement	Source	Priority	Acceptance Criteria
NFR-001	Pages should load within an acceptable time	Quality	Must	Submission pages should load in 3 seconds
NFR-002	User data is protected	Security	Must	No data should be accessible without authentication
NFR-003	Works on standard browsers	Quality	Should	Core features function on the latest two versions of Chrome, Edge

Requirements Traceability Matrix

ID	Stakeholder	Requirement Summary	Acceptance Criteria	Status
FR-001	Requester	Submit a new service request	Saved with unique ID, status "Submitted"	Baselined
FR-002	Requester	Categorise the request	Submission blocked without a category	Baselined
FR-003	Requester	View request status	Status visible and updates in real time	Baselined
FR-004	Requester	View request history	"My Requests" lists past submissions	Baselined
FR-005	Requester	Update/withdraw a request	Editable only while "Submitted"	Baselined
FR-006	Staff	Search/filter/sort requests	Filters return matching requests only	Baselined
FR-007	Staff	View full request details	All fields visible on one screen	Baselined
FR-008	Staff	Accept/assign a request	Accepting sets status to "In Progress", records owner	Baselined; traces from the Staff stakeholder need
FR-009	Staff	Update status via controlled transitions	Only Submitted→In Progress→Resolved→Closed allowed	Baselined
FR-010	Staff	Add notes/comments	Notes timestamped and saved to history	Baselined
FR-011	Management	View activity by status	Counts match live data	Baselined
FR-012	Management	View requests by category	Category filter returns matches	Baselined

FR-013	Role-based access	Restrict view by role	Requesters/Staff/Management see only their scope	Baselined
NFR-001	Quality	Acceptable page load time	No more than 3 seconds should be spent loading the dashboard	Baselined
NFR-002	Security	Protect user data	Passwords hashed, no unauthenticated access	Baselined
NFR-003	Quality	Works on standard browsers	Functions on the latest two Chrom or Edge	Baselined

CONSTRAINTS AND TRADE-OFFs ANALYSIS

Scope Constraints:

- It is important to note that Milestone 1 is restricted to engineering foundations, and the final technology decision, software architecture, database schemas, UI implementation, APIs, and final production deployment are not included.

Schedule & Resource Constraints:

- Under tight milestones and with only 3 team members, it's a real challenge to balance rigorous documentation, risk registries and GitHub governance.

Quality & Security Constraints:

- Non-functional requirements (e.g., page load times less than 3 seconds, strict authentication requirements) should be expected at a high level now.

Trade-Off / Ripple Effect:

- Project scope is kept to a minimum and automated notifications (SMS/email) are not a part of the project the project's scope and cost are maintained as contained.

Initial Risk Register

Risk Item 1 (High Priority):

- Incomplete requirements traceability – causing misaligned acceptance criteria and failing testability in later phases.
- Probability & Impact: Medium probability, High impact.
- Mitigation: Maintain continuous traceability of Requirements Traceability Matrix (RTM) with the needs of the stakeholders.
- Contingency: Do a cross-team audit of RTM prior to baseline sign-off.

Risk Item 2:

- Using AI-based tools too heavily which leads to content that is not verified or compliance issues.
- Probability & Impact: Medium probability, medium impact.

- Mitigation: Require human verification of all AI inputs and add those inputted to the AI Usage Register.
- Contingency: Double check all documentations manually and make sure to delete the unverified text before you submit.

Forward Engineering Considerations Register

- **Register Security & Access Control:**
  Should be taken into account at this stage to prevent having to add security models after the work was completed.
- **Testability:**
  The requirements should be decomposed into testable criteria of acceptance, including the future, in order to have success with automated and manual test framework.
- **Deployment Environment:**
  Knowing where Civic Connect will be deployed helps reduce friction in the deployment process.
- **Data Migration & Recovery:**
  Ahead of the curve with data volume recognition prevents incorrect database schema decisions.
- **Security:**
  Provides security features for future services to meet the security policies of the system.
- **Observability:**
  Planning system logging and audit trails now ensures transparency of operation during future maintenance. The operational cost of assessing resource footprints avoids expensive cloud or infrastructure architectures going down the pipeline.

Engineering Decision Log

Decision 1 (Made):

- Using a protected main branch and a two-reviewer model to ensure version control compliance on GitHub.
- Justification: Enforces project-wide control and auditability per the Master Project Brief.

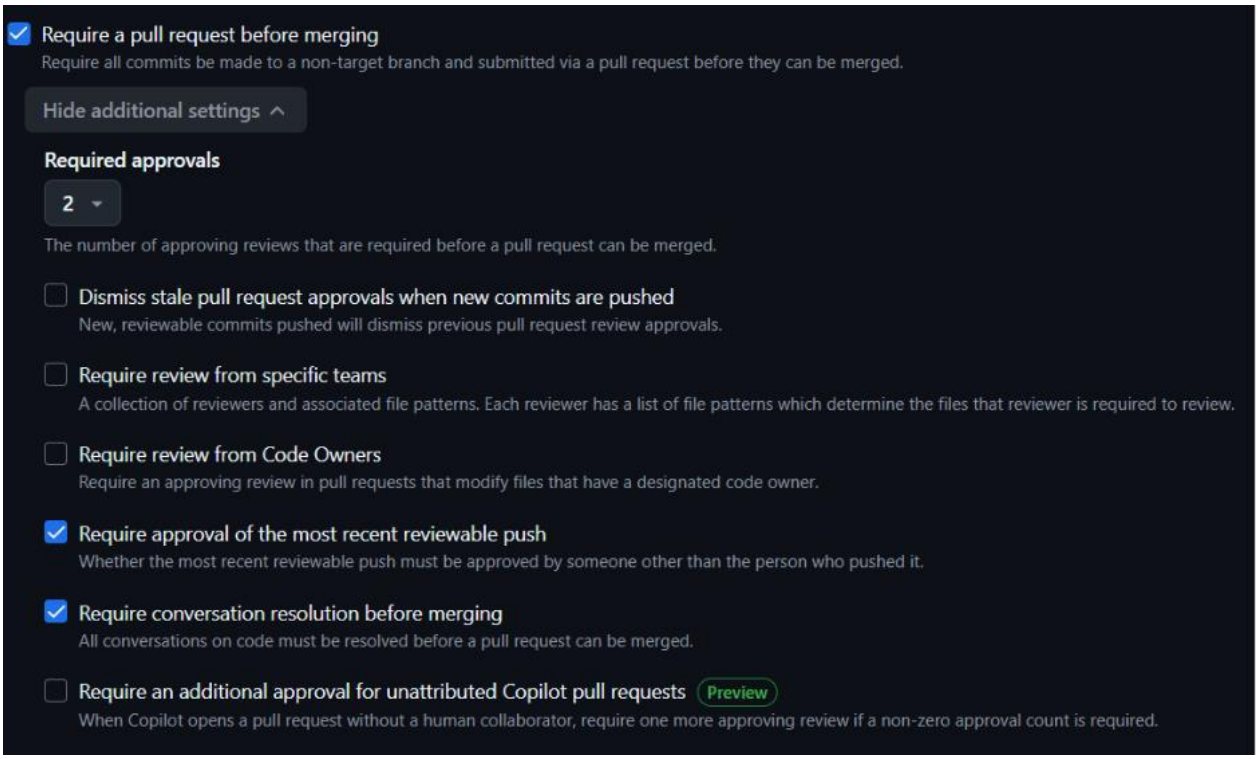
Decision 2 (Deliberately Deferred):

- Deferral of automated notifications (SMS/email) and advanced management reporting dashboards.
- Justification: Focus on the core request lifecycle (submission, assignment, tracking, resolution) is at present; adding third-party integrations adds a lot of unnecessary cost and integration risk.

Repository and Team Governance

Github governance

Github governance refers to using rules and controls to manage the development of a software product. For the project, social coding, which is an agile approach for team development, will be used to allow for a collaborative environment (Marius Golombeck, Orlowski and Tübben, 2018). For the governance of the repository, there will be protection rules for the main branch. According to Github (n.d), a custom rule can be applied to a branch, requiring two or more collaborators to approve a request to merge to the main branch. Using this protection rule will help ensure that every piece of code committed to the main branch is tested before approval. It will also ensure that bulk uploads of content that is unverified will not be done.



Above is the rule set that will be applied for the other milestones of our project.

Team Gorvenance

The following will be done to ensure that at the end, a functional software product that meets the needs and requirements of the stakeholders:

- **Using a protected main branch in the repository**

  This will ensure that teams can work on their parts with full autonomy on separate branches and only commit work to the main branch after review.

- Two collaborators out of the three will have to approve any changes that will have to be merged to our main branch.
- Pull requests with detailed descriptions, branches, and commits will be used throughout the project.

AI Compliance and controls

AI compliance refers to ensuring that AI used adheres to the legal and ethical standards (McGrath and Jonker, 2024). It is also about making sure that any decision taken with the assistance of AI is well-informed and it goes through human review to maintain transparency and trust between the team and the stakeholders (McGrath and Jonker, 2024). For the AI compliance part of our project, we will focus on the following:

- **Planning**

  Before using any AI assisted coding tools, the focus will be on planning the architecture, testing plans, and integration points of the project first (Sreejith Sreejayan, 2025). Then a plan on how to handle AI generated output will be handled.

- **Considering constraints during development**

  Instead of just sending prompts to AI models, during development, constraints will be sent to the AI models to assist in building individual modules to prevent multiple errors caused by prompting AI to code everything (Sreejith Sreejayan, 2025).

- **Prioritizing the understanding of the functionality of the code**

  The team will focus on understanding the implementation of the code, the data flow in the system, and how to debug code when there might be a requirement change (Sreejith Sreejayan, 2025).

- **Security focused AI assisted development**

  All AI code must be treated as potentially malicious as up to 35.8% of AI generated code can cause security weaknesses in software systems (Sreejith Sreejayan, 2025). As a precaution the following rules will be followed when developing the software system:

  - API keys or any sensitive information should not be added into any AI prompts.
  - Placeholder values should be used in prompts instead of using real values.
  - AI generated code should not be deployed without human review.

- **AI usage register**

Throughout the project, we will be using an AI usage register to document all the AI outputs that will be used in the project. This will include a unique ID for each output, its source, purpose, owner, and risk.

Milestone 2: Architecture, Technology & Initial Design Baseline

Architecturally Significant Requirements (ASR)

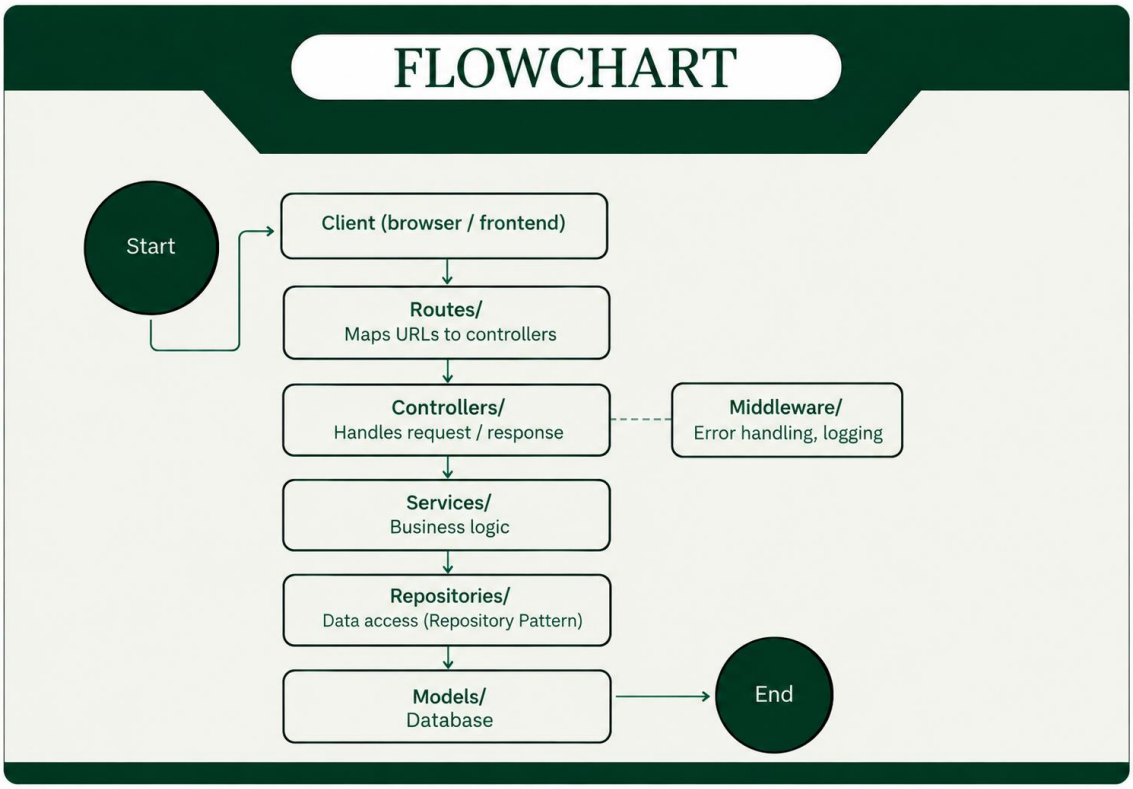
- **ASR-01: Role-Based Access Control**
  Linked to: FR-013, NFR-002, Stakeholder Analysis
  Requesters, Staff, and Management must see only data relevant to their role, and no data may be accessible without authentication. This drives a clear separation between the layer that handles authorization and the layer that contains business logic, so access rules aren't scattered across the codebase and can be enforced consistently at the API boundary.

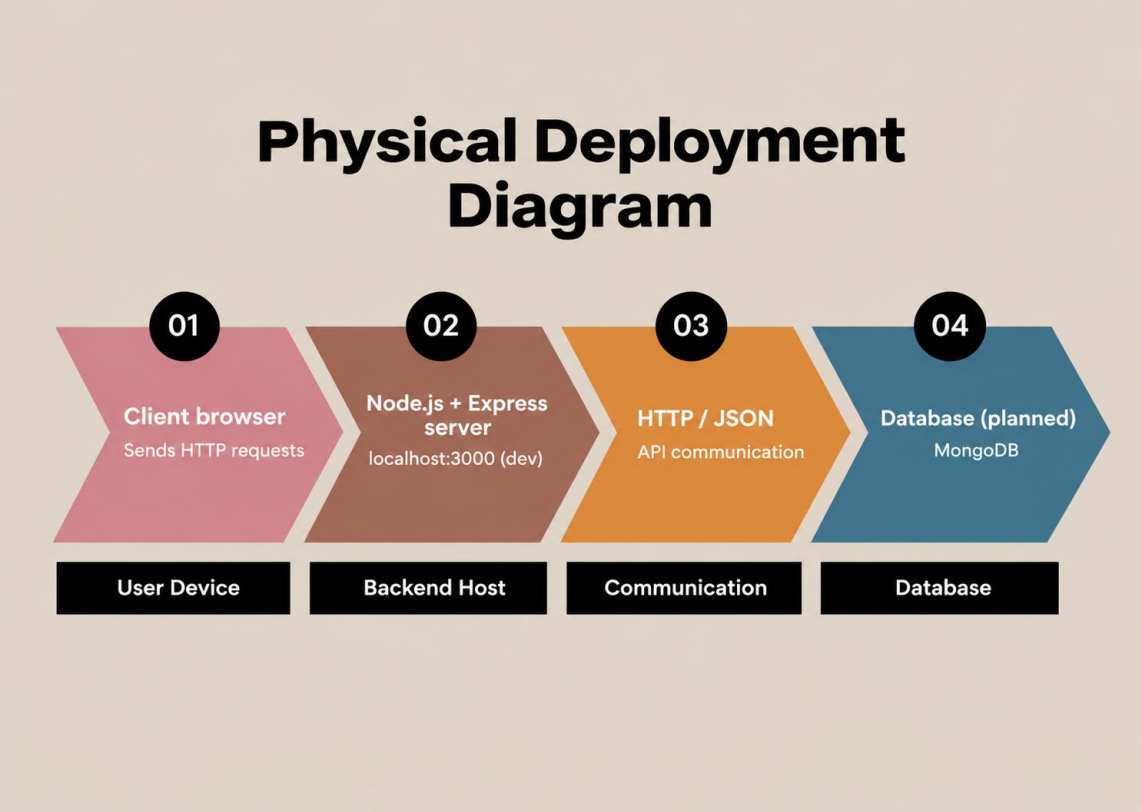
- **ASR-02: Consistent, Timely Status Visibility**
  Linked to: FR-003 (status visible to Requester in real time as Staff changes it)
  When Staff update a request's status, the Requester's view must reflect that change reliably. This encourages a request-response approach where each status read encounters the same authoritative data stream and necessitates that the architecture maintains a single source of truth for request state (avoiding duplicate or cached copies of status that could get stale).

- **ASR-03: Data Integrity Across the Request Lifecycle**
  Linked to: FR-009 (controlled status transitions), FR-010 (timestamped notes), FR-011/012 (Management reporting)
  Actions must be timestamped and auditable for management reporting, and status changes must occur in a predetermined order (Submitted → In Progress → Resolved → Closed). To prevent improper transitions from occurring and to ensure that every change has a single, traceable cause, business logic must be centralized rather than replicated across UI and API.

- **ASR-04: Proportional Complexity for Team Size and Schedule**
  Linked to: Constraints and Trade-Offs Analysis — Schedule & Resource Constraints
  With a three-person team and a tight academic schedule, the architecture must be simple enough for all members to understand, extend, and defend individually, while maintaining clear separation of concerns. A single, well-organized codebase is therefore more appropriate than a distributed architecture.

Logical architecture and physical deployment diagram





ADR 01: Architecture Pattern Selection

Context

CivicConnect needs an architecture that supports role-based access control, consistent real-time status visibility, and centralized, auditable business logic for requesting lifecycle transitions. At the same time, the team consists of three students working under a tight academic schedule, with no prior experience operating distributed systems in production.

Decision

We selected a layered monolithic architecture (single deployable Node.js/Express application, internally organised into routes, controllers, services, repositories, and models) over a microservices architecture.

Alternatives Considered

Alternative	Why rejected
Microservices	CivicConnect's small team and scope do not justify the added complexity of independent deployment pipelines, network boundaries, and inter-service connectivity. Section 5.3 also states that a more distributed architecture is not necessarily more advanced, making a

	monolithic approach more suitable for the team.
Unstructured monolith	A single-layer design would keep deployment simple but would fail ASR-01 and ASR-03, as Section 5 highlights the need for separated layers to ensure access control and business rules are applied consistently and remain auditable.

Why the layered monolith fits

The layered monolith fits CivicConnect because authentication and role checks are handled centrally in the middleware and controller layers, satisfying ASR-01. Requests follow the same service and repository layers to one authoritative data source, satisfying ASR-02. Business rules, including status transitions, are centralized in the service layer, ensuring consistent validation and satisfying ASR-03. A single codebase and deployment target also suit the three-person team and one semester timeline, satisfying ASR-04.

Consequences

- Benefit: simpler local development, testing, and deployment.
- Trade-off: Individual components, such as a future notifications function, cannot be scaled separately. This is acceptable since advanced scaling and notifications are out of scope, as stated in the M1 Scope Baseline and Deferred Scope.
- Forward consideration: If a future phase required independent scaling of a specific feature, the layered structure's separation of concerns would make it easier to extract that feature into its own service later than if the code had started unstructured.

ADR 02: Technology Stack Selection

Context

CivicConnect requires a backend runtime and framework capable of handling role-based routing, JSON-based request/response cycles, and centralised business logic, while remaining within the team's existing skill set and the constraints of a single-semester academic project with three developers.

Decision

We selected Node.js as the runtime and Express.js as the backend web framework.

Alternative	Why rejected
.Net/ ASP.NET Core(C#)	Like ADR 01, this approach was excluded because its heavier conventions, including

	compiled builds, strongly typed interfaces, and an integrated DI container, add unnecessary complexity for a three-person team building the architecture from scratch in a single semester.
Python/Django	Django's built-in structure is more opinionated than needed for CivicConnect and would require the team to learn a new language without offering a significant architectural benefit over Express.

Why the Node.js and Express fits

The proportionality principle behind ADR 01, that added complexity must be justified by project needs, also applies to the language and framework choice. ASP.NET Core supports layered architecture but introduces additional setup through compiled builds, strongly typed interfaces, and dependency injection. While valuable for larger systems, this adds unnecessary complexity for a three-person team building the architecture from scratch in a single semester. Express is more lightweight and unopinionated, allowing the team to implement the routes, controllers, services, and repositories directly without adapting to rigid framework conventions. Combined with the team's existing JavaScript familiarity, this allowed more effort to focus on development rather than learning a new language and framework.

Deployment Direction

CivicConnect will run as a single Node.js/Express application serving a browser-based client, backed by one database, on a conventional web hosting environment. All environment-specific settings are held in environment variables, so the application is not tied to one provider. This is compatible with the layered monolith in ADR 01 because there is only one deployment unit to host, and with the stack in ADR 02 because Node.js is widely supported by hosting providers.

Deliberately deferred: the hosting provider and production configuration. Evidence still needed: the institution's hosting options (if any), cost limits, and the database engine chosen in ADR 03.

ADR 03: Data and Persistence Store Selection

Context

CivicConnect will need persistence data storage that will work with Node.js backend to store the application information such as end user account details, service requests and audit logs. The database should have strict state transitions and maintain the integrity of the data when updates occur (ASR-03), and it will support database queries made by users in different departments (ASR-01).

Decision

For CivicConnect, MongoDB that will be accessed via Mongoose ODM was chosen as the database.

Alternatives Considered

Alternative	Why was it rejected?
PostgreeSQL	Using Postgree requires building the table schemas and ORM mapping layers. All of this adds more structural overhead.
In-memory storage	Using In-memory storage leads to the database to lack transactional integrity, indexing abilities and query capabilities that will be needed for data reporting.

Why MongoDB is fit for the project

- MongoDB is more compatible with JavaScript, eliminating any need for database conversion between API controllers in the system.
- Mongoose allows the team to define the database tables, schemas, and other required properties for entities inside JavaScript code.
- Mongoose will support the MongoDB session transactions that will make sure that there are state changes and audit logs to satisfy ASR-03.

Consequences

Using MongoDB will benefit CivicConnect by providing a fast development environment for the database and simple compatibility with the Node.js tech stack. Although using MongoDB has may benefits there is a trade-off, the integrity of the data is enforced using explicit application log.

Database Schema Specification Tables

1.User

Field Name	DataType	Constraints	Description
_id	ObjectId	Primary key that is automatically generated	Unique identifier for User
name	String	Required, trim	Name of account holder
email	String	Required, unique, lowercase	Main email for log in
password	String	required	Account password
role	String	Required, Enum [“resident”, “staff”,	Account restriction is needed to prevent

		”supervisor”, ”admin”] , Default: null	unauthorized access
departmentId	String	Default: null	Specifies the department of user is relevant for access
createdAt	Date	Timestamps: true	Automatically saves the time the account was created
updatedAt	Date	Timestamp: true	Automatically saves the time the account was updated at

2. Request

This is the table for the requested services by the users

Field name	DataType	Constraints	Description
_id	ObjectId	Primary key automatically generated	Unique identifier for a request
title	String	Required, trim	Summary of the service requested
description	String	required	Detailed explanation of the reported issue
status	String	Required, Enum: [‘OPEN’, ‘IN_PROGRESS’, ‘PENDING_INFO’ , ‘RESOLVED’ , ‘CLOSED’, ‘CANCELLED’]	Describes the status of the request lifecycle.
departmentId	String	Required	Specifies the department of a worker
residentId	ObjectId	Required, reference: User	Foreign key that links the resident account to the users
assignedStaffId	ObjectId	Default: null, Reference: User	Foreign key that links the assigned staff member to the User
createdAt	Date	Timestamp: true	Saves date of the creation of a request
updatedAt	Date	Timestamp: true	Saves date of the update of a request

3. AuditLog

Field Name	DataType	Constraint	Description
_id	ObjectId	Primary key that is automatically generated	Unique identifier
requestId	ObjectId	Requested, Reference: Request	Foreign key to reference the service requested
oldStatus	String	required	The status of the service requested before any changes
newStatus	String	Required	New status after changes
changedBy	ObjectId	Required, Reference: User	Foreign key referencing a user account
timestamp	Date	Default: Date.now	This is the timestamp when status transmission is logged

Design Pattern Architecture Decisions

ADR 04: Repository Pattern Implementation

Category

Structural

Context

The structural design pattern will be used to minimize any queries from being leaked into the app controllers or any business logic. Another benefit is that data logic must be encapsulated.

Decision

We will have a controller that will call the service methods, and the query Mongoose models through the repository.

Consequences

It will make sure that Mongoose queries are reusable in different services throughout the application.

ADR 05: State machine

Category

Behavioral Design

Context

Considerations for forward engineering need a audit trail when a status changes. The service requests must progress through controlled status transitions.

Decision

We will use a Finite State Machine transition matrix in conjunction with the Mongoose model.

Request Transition Matrix

Status	Allowed Target Status	Authorized role
OPEN	IN_PROGRESS, CANCELLED	Staff, supervisior, resident
IN_PROGRESS	RESOLVED, PENDING_INFO, CANCELLED	Staff, supervisor, admin
PENDING_INFO	IN_PROGRESS, CANCELLED	Staff, supervisor, resident
RESOLVED	CLOSED	Staff, supervisor, admin
CLOSED	None	None
CANCELLED	None	None

Updated Risk Register

Risk Item 3: Layer boundaries erode as the team adds code

- Description: With three members building in parallel inside one codebase, business logic could leak into controllers or database calls into services, undermining the separation that ASR-01 and ASR-03 depend on.
- Probability & Impact: Medium probability, High impact.
- Mitigation: Pull request reviews explicitly check that each change stays within its layer, and the repository pattern skeleton gives the team a concrete example to follow.
- Contingency: Refactor the offending code before merging; if it recurs, add a short layer-rules checklist to the pull request template.

Risk Item 4: Authentication and role checks are not yet implemented

- Description: NFR-002 and FR-013 require that no data is accessible without authentication, and that each role sees only its own scope. The current bootstrap has no authentication, so any endpoint added now is open by default.
- Probability & Impact: High probability, High impact.
- Mitigation: Build authentication and role-checking as middleware early in M3 construction, before feature endpoints multiply. Until then, no endpoint handling real user data is to be exposed.
- Contingency: If authentication slips, restrict development to local environments only and

block deployment until it is in place.

Risk Item 5: Dependency and ecosystem vulnerabilities

- Description: The application relies on third-party npm packages, which can carry security flaws or become unmaintained.
- Probability & Impact: Medium probability, medium impact.
- Mitigation: Run npm audit regularly, keep dependency versions recorded in package.json, and add only packages the team can justify.
- Contingency: Replace or upgrade any package with a known high-severity vulnerability before the next baseline.

Risk Item 6: Secrets or configuration committed to the repository

- Description: Environment values such as database credentials could be accidentally pushed to GitHub.
- Probability & Impact: Low probability, High impact.
- Mitigation: The real environment file is excluded through .gitignore, only a placeholder example file is committed, and pull request reviewers check for credentials.
- Contingency: If a secret is ever committed, rotate it immediately and remove it from repository history.

Risk Item 7: Deployment provider not yet selected

- Description: The working direction is a single Node.js application with its own database on a conventional web host. The provider has not been chosen, so configuration, networking, and persistence details may need to be adjusted.
- Probability & Impact: Medium probability, medium impact.
- Mitigation: All environment-specific values live in environment variables, and the database and frontend choices are made with common hosting options in mind.
- Contingency: If the chosen host has incompatible requirements, adjust configuration rather than architecture, since the monolith has a single deployment unit.

Forward Engineering Considerations

- **Authentication mechanism:** We have not yet decided how users will log in, for example with sessions or tokens. This needs its own decision to record before we build it, because it determines how role-based access is enforced under ASR-01. Until then, no endpoint should expose real user data.
- **Automated testing:** No testing framework is set up yet. We should choose one before much

business logic is written, so the M1 acceptance criteria can become automated checks.

- **Logging and audit trail:** Errors are currently only logged into the console. Proper logging and audit trails will be needed for FR-010 and Management reporting, so this is deferred until the database layer exists.
- **Database backup and recovery:** We plan to use the MongoDB Atlas free tier during development. Before relying on it, we must confirm its backup options and that its storage and transfer limits cover our expected use, using MongoDB's official documentation. This links to Member 2's ADR 03.
- **Data integrity for status changes:** The status flow from Submitted to Closed must be protected in the database as well as in the service layer, so two staff members cannot make conflicting updates to the same request. Member 2's persistence work will define how we do this.
- **Server hosting:** The hosting provider for the Node.js server is still open. We still need to find out which options are available to us, what our cost limits are, and whether the institution provides any infrastructure. Keeping all settings in environment variables means we can change providers without changing the code.
- **Extracting services later:** If notifications, which we deferred in M1, come back into scope, the layered structure lets us move that feature into its own service without rewriting the rest of the system.

Architecture, Technology & Initial Design Baseline

Baseline identification

This is the Milestone 2 baseline for CivicConnect. It is part of PED version 2.0 and is dated 28 September 2026. It is the set of architecture, technology, and design decisions the team has agreed on, so we can start building without changing our minds every week.

The baseline includes:

- the architecturally significant requirements (ASRs)
- the logical and physical architecture diagrams
- ADR 01 (architecture pattern) and ADR 02 (technology stack)
- the deployment direction
- the data and persistence decisions
- the initial design pattern decisions
- the updated risk register and forward engineering considerations

- the backend code on the backend-bootstrap branch

What we have agreed on

CivicConnect will be a layered monolith built with Node.js and Express, using MongoDB as the database. Every one of these decisions links back to our M1 requirements and the ASRs

What is still open

Some things are deliberately left for later. These are the authentication methods, the testing framework, the server hosting provider, and the final logging approach. Each one is listed in the Forward Engineering Considerations, together with what we still need to find out.

How changes are handled

If we need to change anything in this baseline, we will write a new or updated ADR and submit it through a pull request that needs two approvals. Old decisions will stay in the document with the reason they changed, so nothing gets overwritten.

Sign-off

Name	Role	Approval	Date
Ayanda Mabena	Architecture, Technology Stack and Governance Lead	Approved	28 September 2026
Refilwe Segele	Team member	Approved	28 September 2026
Nompilo Mbense	Team member	Approved	28 September 2026

Milestone 1 feedback and evidence

1. **Performance requirement (NFR-001)**

Our three-second target for submission pages is based on research into similar web products. Google found that more than half of mobile visits are abandoned when a page takes longer than three seconds to load (Sullivan, 2016). This matters for CivicConnect because a requester who gives up on a slow page will go back to phone calls or WhatsApp, which is the exact problem the system is meant to solve. Our layered Express application with a single database keeps pages small, so we believe three seconds is realistic.

2. **Out-of-scope: native mobile app**

Native mobile app development is out of scope because it would require separate, platform-specific applications for systems such as Android and iOS. Native apps also require platform-specific tools and development to access device features such as GPS, cameras and Bluetooth (Singh, 2023). Since CivicConnect's current scope does not require these capabilities, a browser-based

application is sufficient and avoids the additional development effort of supporting separate native apps.

3. Deferred scope: SMS Notifications

SMS notifications were deferred because SMS is a relatively expensive channel and is generally recommended for critical or time-sensitive messages (Chris, 2025). Supporting SMS would also require additional components, including an SMS worker and external SMS gateway (Chris, 2025). Since this functionality is not essential to CivicConnect's current scope, deferring it avoids unnecessary cost and complexity while allowing the team to focus on the system's core requirements.

CONCLUSION

By enforcing the following branch protection rules, review approval rules, and structure for pull requests and commits, accountability and transparency throughout the project will be maintained. These rules along with AI compliance will ensure that AI tools will be used to further improve the project and to prevent any defects in the software project. These will ensure that the integrity of the software project in this milestone through to the last milestone.

References

ServiceNow, 2026. *ServiceNow.* [Online] Available at: https://www.servicenow.com/products/itsm/what-is-service-request-management.html [Accessed 8 September 2026].

Belgium Campus ITversity, 2026. *SEN381 Software Engineering 381: CivicConnect Master Project Brief.* Pretoria: Belgium Campus ITversity.

Belgium Campus ITversity, 2026. *SEN381 Project 1: Requirements, Traceability, and Scope.* Pretoria: Belgium Campus ITversity.

Somerville, I., 2015. *Software Engineering.* 10th ed. Boston: Pearson.

Pressman, R.S. & Maxim, B.R., 2020. *Software Engineering: A Practitioner's Approach.* 9th ed. New York: McGraw-Hill Education.

ServiceNow, 2026. *ServiceNow.* [Online]
Available at: https://www.servicenow.com/products/itsm/what-is-service-request-management.html
[Accessed 8 september 2026].

Atlassian (2024). *What is version control*. [online] Atlassian. Available at: https://www.atlassian.com/git/tutorials/what-is-version-control [Accessed 8 Sept. 2026].

David Rendon (2026). *How GitHub enterprise governance lets you ship secure code at scale without slowing down your…*. [online] Medium. Available at: https://medium.com/codex/how-github-enterprise-governance-lets-you-ship-secure-code-at-scale-without-slowing-down-your-90b83a0dba2f [Accessed 8 Sept. 2026].

Github (n.d.). *About protected branches*. [online] GitHub Docs. Available at: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches [Accessed 8 Sept. 2026].

Marius Golombeck, Orlowski, H. and Tübben, S. (2018). *Collaborative Software Development with GitHub® Projects*. [online] Available at:

https://www.researchgate.net/publication/330162832_Collaborative_Software_Development_with_GitHubR_Projects [Accessed 8 Sept. 2026].

McGrath, A. and Jonker, A. (2024). *AI compliance*. [online] Ibm.com. Available at: https://www.ibm.com/think/insights/ai-compliance [Accessed 9 Sept. 2026].

Sreejith Sreejayan (2025). *How to use AI for your software project: A complete guide for 2025*. [online] Medium. Available at: https://medium.com/@the-sreejith/how-to-use-ai-for-your-software-project-a-complete-guide-for-2025-3dda56c3fb23 [Accessed 9 Sept. 2026].

Chris, G. (2025). *Designing a Notification System: Push, Email, and SMS at Scale*. [online] DEV Community. Available at: https://dev.to/sgchris/designing-a-notification-system-push-email-and-sms-at-scale-kio [Accessed 29 Sept. 2026].

Singh, H. (2023). *Native mobile app development: Pros, cons, challenges & alternatives*. [online] Debut Infotech. Available at: https://medium.com/debutinfotech/native-mobile-app-development-pros-cons-challenges-alternatives-5b42814f610b [Accessed 29 Sept. 2026].

Sullivan, L. (2016). *Many visitors abandon mobile sites if load time tops 3 seconds*. [online] mediapost. Available at: https://www.mediapost.com/publications/article/284398/many-visitors-abandon-mobile-sites-if-load-time-to.html [Accessed 29 Sept. 2026].